IITM PRAVARTAK
CATALYSING INNOVATION

Professional Certificate Programme in Agentic AI and Applications

Agent Evaluation and Debugging

Revisit the Learning So Far

Let’s take a quick look at the key ideas and concepts you have explored in your learning journey.

- How agents are deployed and monitored in real-world environments
- Real-world case studies showcasing agentic system implementation
- How hosting choices influence agent performance and scalability
- How APIs enable agent deployment using FastAPI frameworks
- Core trade-offs between different hosting strategies for AI agents
- How agents are deployed across varied infrastructure setups
- How monitoring and observability improve agent reliability and transparency
- How reflection mechanisms enhance agent learning and adaptability

Topics Covered

- Evaluating and Monitoring Agentic Systems
- Multi-Dimensional Metrics
- Design a Custom Evaluation Rubric
- Case Review
- Debugging with Logs
- Reflection & Wrap-Up

Evaluating and Monitoring Agentic Systems

From Observability to Debugging – Hands-on with LangSmith & Logs

What You'll Master Today

Agent evaluation challenges

Understand the unique complexities of multi-step reasoning systems

Key metrics framework

Track success rates, coherence, correctness, coverage and latency

Tracing v. logging

Compare structured observability with LangSmith to manual approaches

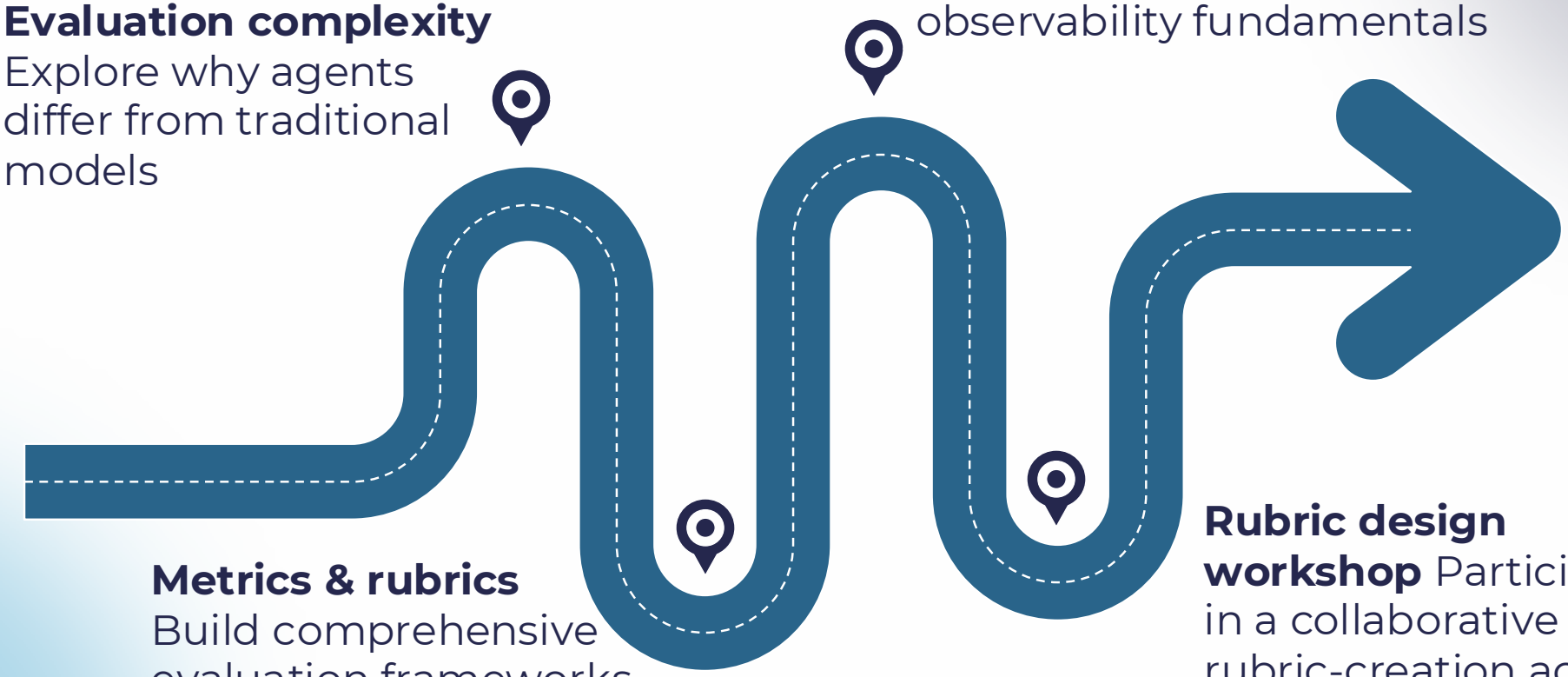
Debug & optimise

Use practical techniques to diagnose issues and improve agent performance

Session Roadmap

Evaluation complexity
Explore why agents differ from traditional models

Logging & tracing demo
Practice hands-on observability fundamentals



Metrics & rubrics
Build comprehensive evaluation frameworks

Rubric design workshop Participate in a collaborative rubric-creation activity

Session Roadmap

Failure analysis
Review cases and diagnose underlying issues

Debugging session
Apply advanced log-analysis techniques

Reflection & takeaways
Synthesise learning and define next steps

Traditional Models v. Agentic Systems

ML models	Agentic systems
Single prediction output	Multi-step reasoning chains
Static evaluation: accuracy, F1, precision	Dynamic metrics: coherence, tool success
Deterministic behaviour	Non-deterministic paths
Clear failure modes: overfitting, bias	Complex failures: hallucination, tool errors

Challenge: The RAG Dilemma

> Your RAG assistant sometimes provides factually correct information from completely irrelevant sources. Why is this difficult to evaluate?

Activity 1: Case Study Discussion

Consider: correctness ≠ relevance, retrieval quality vs answer quality, multi-dimensional evaluation requirements

Multi-Dimensional Metrics

Evaluating agents requires **comprehensive frameworks** that measure quality, coverage and efficiency across the entire reasoning chain.

Metric 1: Success Rate

- **Definition**
  Percentage of queries where the agent produces a valid final answer
- **Formula**

$$Success\ Rate = \frac{Successful\ Completions}{Total\ Queris} \times 100\%$$

Example evaluation: Agent completed 87 out of 100 queries successfully with a **success rate 87%**

Metric 2: Coherence & Correctness

- **Coherence:** Logical flow and consistency between reasoning steps
- **Correctness:** Agreement with ground truth, factual accuracy of outputs

Key insight: An agent may be coherent yet wrong or correct yet incoherent—both dimensions matter.

Metric 3: Coverage & Latency

Coverage	Latency
Agent's ability to handle diverse query types and edge cases	**Average time to completion (TAT - turnaround time)**
Query type distribution Edge case handling Graceful degradation	Per-step timing Total response time Performance bottlenecks

Real-World Metrics Dashboard

LangSmith provides comprehensive per-run visibility

Run ID	Success	Coherence	Correct	Latency	Token Count
run_4a2f	✓	4.2/5	✓	1.8s	287
run_8b1c	X	2.1/5	X	3.2s	445
run_5d9e	✓	4.7/5	✓	1.5s	312
run_2c8a	✓	3.9/5	✓	2.1s	398

Observability for Agents

- **Logging:** Event recording for post-hoc debugging and analysis
- **Tracing:** Structured and hierarchical representation of agent execution steps

Goal: Complete visibility into agent decision-making and execution paths

LangSmith: Observability Platform

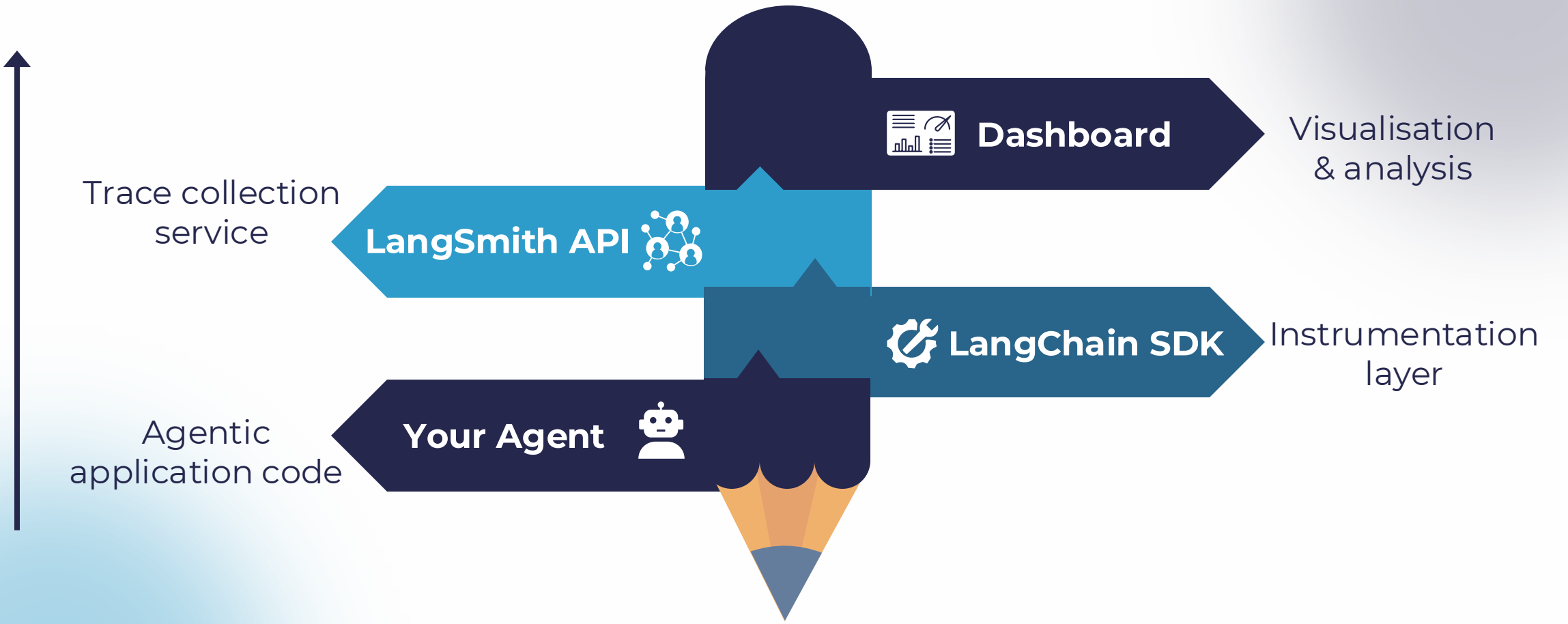
Full visibility: Tracks inputs, outputs and intermediate steps for every LangChain application run

Performance metrics: Captures latency, token usage and cost per execution automatically

Error detection: Delivers real-time alerts and detailed error traces for rapid diagnosis

Team collaboration: Provides shared dashboards and project-level insights across your organisation

Architecture: Agent to Dashboard



Automatic instrumentation with minimal code changes required

Tracing > Print Debugging

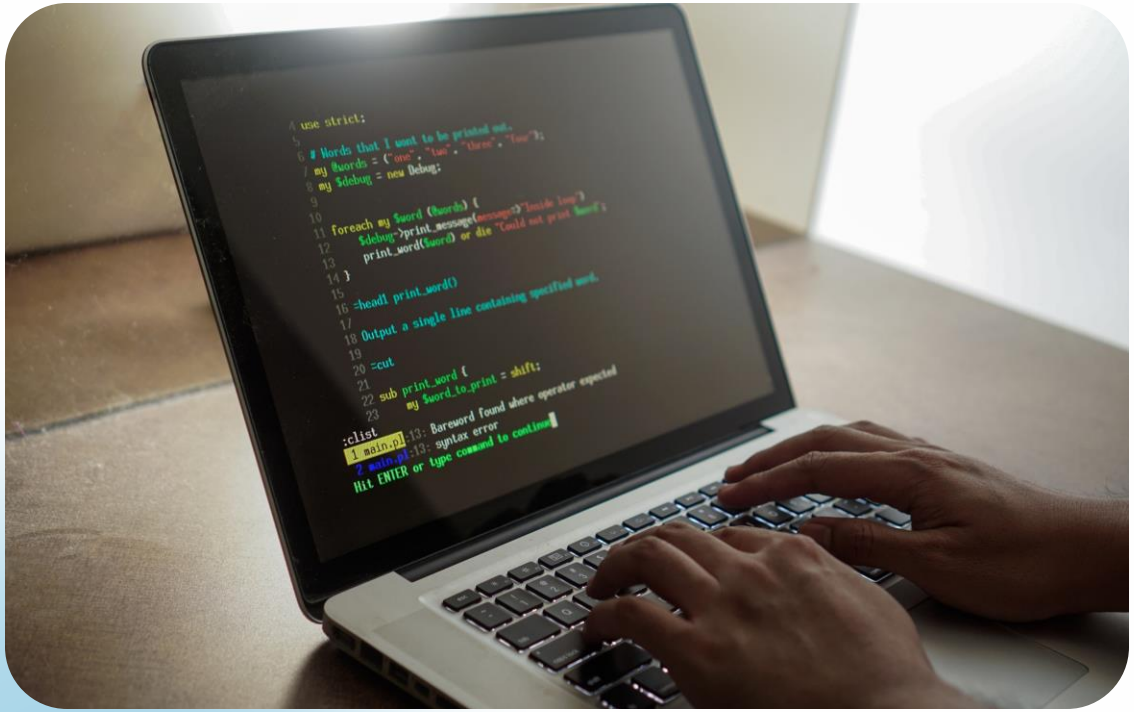
✗ Print Logs

- Scattered console output
- No execution context
- Local debugging only
- Manual metadata tracking
- Difficult to share

✓ Structured Tracing

- Hierarchical step view
- Full chain context
- Team visibility
- Automatic metadata
- Searchable & filterable

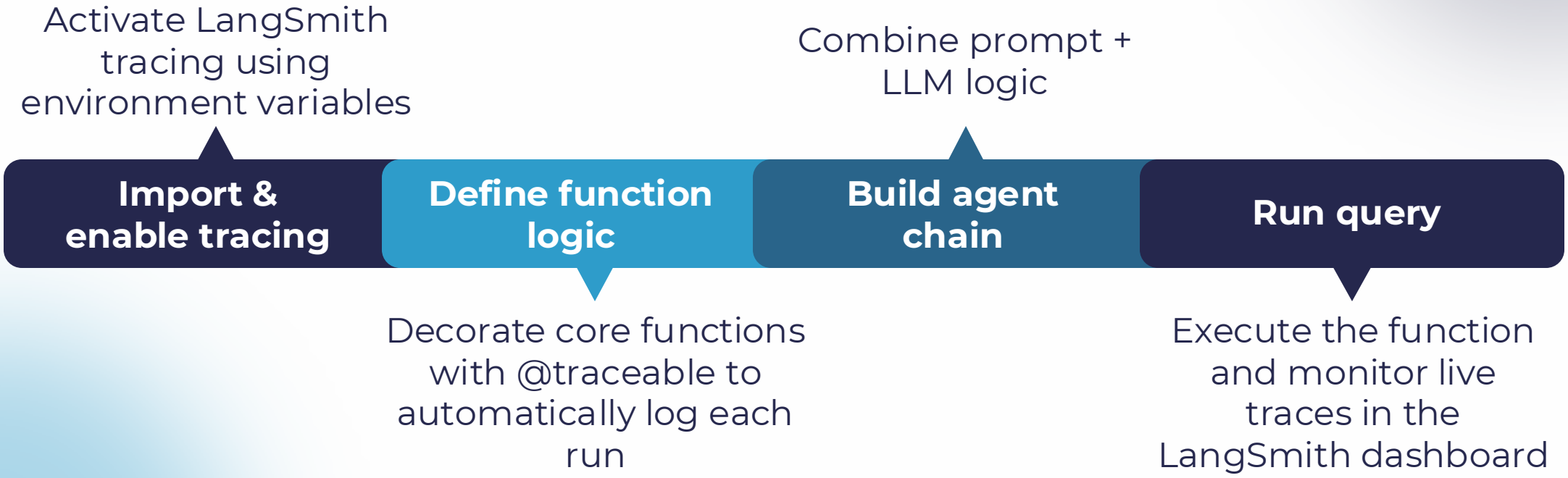
Demo 1: Logging & Tracing Basics



- **Environment setup:** Refer to *`langsmith_tracing_demo\README.md`*
- **Expected output:** Live trace URL in console, real-time dashboard updates, detailed execution tree with timing

Implementation Flow

Implementation Steps Involved



Demo Insights: What You'll See

Create a unique trace for each query with full execution history visible in the LangSmith dashboard

Record prompt text, model output, latency data and token usage at every step

Display success trends, error patterns and performance bottlenecks through dashboard visualisations

Automatic trace creation

Comprehensive logging

Visual analytics

Ready to Build Better Agents?

4

Core metrics
Success, coherence, correctness, latency

2

Observability layers
Logging and structured tracing

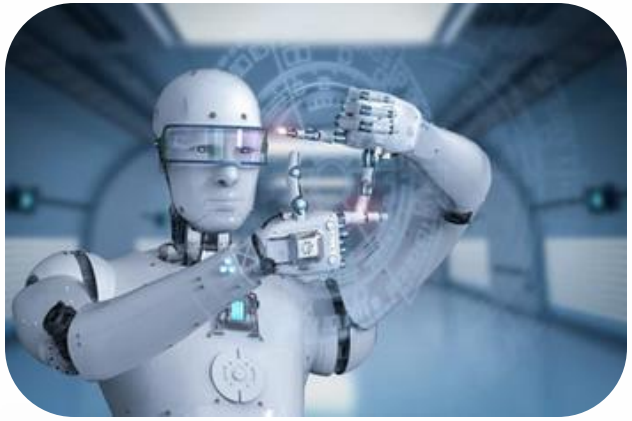
100%

Visibility
Complete agent execution transparency

Design a Custom Evaluation Rubric

Build robust evaluation criteria tailored to your agent's unique requirements and use cases

Activity



Analyse evaluation needs

- Reflect on what makes an agent “high quality.”
- Think about aspects like success, coherence, correctness, latency and user satisfaction

Design rubric

- Create your own evaluation framework
- Define 4–5 key criteria and describe what “Excellent,” “Good” and “Needs Improvement” look like

Present & share

- Share your rubric and explain the reasoning for chosen metrics and thresholds

Rubric Template Structure

Metric	Description	Score (1–5)	Weight
Coherence	Logical reasoning flow throughout response	—	—
Correctness	Accuracy of facts and information provided	—	—
Tone	Politeness, helpfulness, user-friendliness	—	—

Use this framework as a starting point and tailor the metrics to your agent's specific goals.

Case Review

Analyse real-world agent failures to strengthen evaluation and debugging skills

RAG Agent Failure Scenario

- **The problem:** The agent retrieved and used irrelevant context when answering the query
- **User impact:** The response quality degraded significantly and delivered incorrect information
- **Root cause unknown:** The retrieval-generation pipeline introduced multiple potential failure points

Diagnostic Questions

Was retrieval accurate?

Did the vector search return documents relevant to the user intent?

Did prompt context match the query?

Was the retrieved context correctly formatted and aligned with the query?

Were hallucinations present?

Did the model generate information beyond what was provided in the context?

Systematic diagnosis reveals where the agent pipeline broke down

Mapping Failures to Metrics

Each failure type links to specific evaluation metrics—use the mapping to drive targeted improvements.

Failure type	Impacted metric
Wrong source retrieved	Coverage / Correctness
Slow response time	Latency
Irrelevant context used	Relevance / Coherence
Hallucinated information	Faithfulness / Accuracy

Debugging with Logs

Manual log analysis remains essential—even with advanced tracing tools like LangSmith

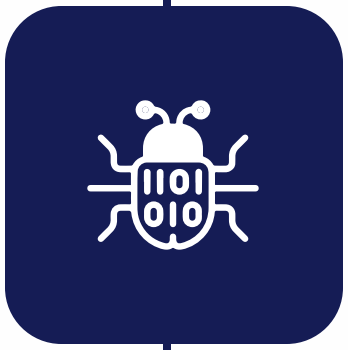
Why Manual Logs Still Matter

Lightweight & fast: Use logs without external dependencies and access them instantly during development

Granular control: Capture only the details that matter for your specific debugging needs

Privacy & security: Keep sensitive data local with no external services involved

Simple debugging: Iterate quickly by adding print statements, running, analysing and fixing without complex setup



Demo 2: Debug a Broken Agent

Setup

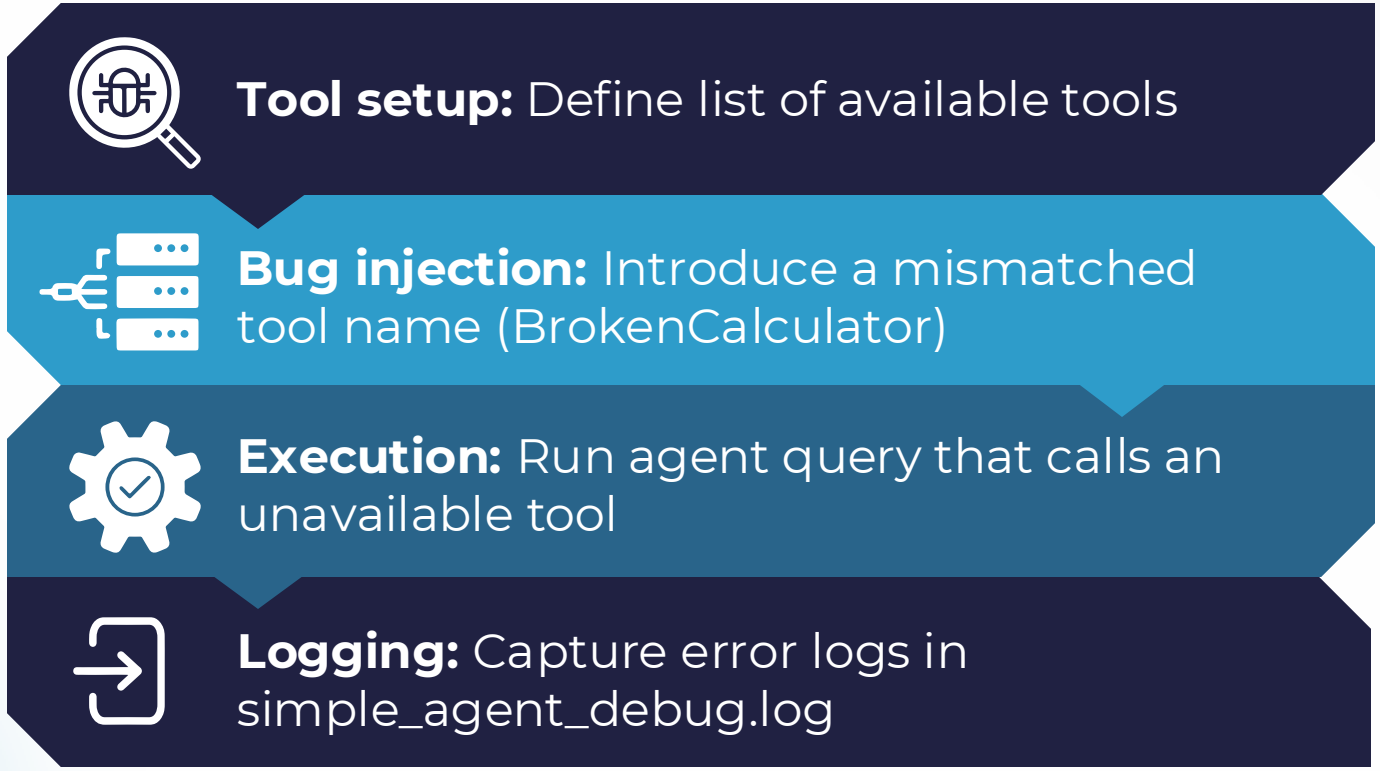
- Simple agent that contains intentional bugs
- Gradio UI that enables user interaction
- Log file: `simple_agent_debug.log`

Debugging Process

- Observe agent failure in UI
- Examine log file output
- Identify root cause from logs
- Apply fix and verify resolution

Debug Logic Overview

Debug Flow



Refer to `simple_agent_debug_demo/README.md`

Log Analysis

Real Log Output from Broken Agent Execution

```
2024-01-15 10:23:41 | INFO  | Query received: "What is 25 * 4?"2024-01-15 10:23:41 | INFO  |
    Available tools: ['Calculator', 'WebSearch']2024-01-15 10:23:41 | INFO  | Trying tool:
    BrokenCalculator2024-01-15 10:23:41 | ERROR | Tool 'BrokenCalculator' not found2024-01-15 10:23
    :41 | ERROR | Agent execution failed2024-01-15 10:23:41 | INFO  | Query completed in 0.01s
    (failed)
```

Key insight: Log clearly shows tool name mismatch—agent requested unavailable tool

Fix & Rerun

Before (Broken)

```
selected_tool =
"BrokenCalculator"# Log
output:ERROR | Tool
'BrokenCalculator'
not found
```

After (Fixed)

```
selected_tool =
"Calculator"# Log
output:INFO | Tool
'Calculator' executedINFO
| Result: 100
```

Resolution: Correcting tool name from "BrokenCalculator" to "Calculator" immediately resolves the failure

Reflection & Wrap-Up

Connect today's concepts to your own agent development workflows

Reflection Exercise

"How can you evaluate and improve agents in your role?"

Individual reflection
Take 2-3 minutes to think about your specific use case

Write your response
Share in chat or document in your notebook

Group discussion
Volunteer to share insights with the class

Key Takeaways

Agent ≠ model evaluation

Agents require multi-dimensional assessment beyond model accuracy alone

Dual debugging tools

Use LangSmith tracing for visualisation + manual logs for granular control

Custom rubrics

Qualitative assessment frameworks capture nuanced agent performance aspects

Continuous monitoring

Ongoing evaluation drives iterative improvement → smarter, more reliable agents

Q & A

Thank you